Java OOP: Cheat Sheet & Exam Explanations

OOP-Java, Dr. Soumaya Trabelsi, PRISTINI 2026-2027 (BI2 / GLSI2). Written from your course slides (slides 1–77) and Labs 1–3.
Every Java example in this sheet was compiled with `javac` before going in the PDF.

Contents: 1. Intro (JVM, JDK) · 2. Ch1 basics: types, variables, operators · 3. Control structures & Scanner · 4. Ch2 class, object, attributes, methods · 5. Constructors & destruction (GC) · 6. `this` · 7. Access modifiers & encapsulation · 8. Scope of variables · 9. Overloading · 10. `static` & `final` · 11. Lab patterns (objects inside objects, arrays of objects, `Math.random`) · 12. Definitions to learn by heart · 13. Compile errors she loves · 14. Errata in the slides

🎯 The words she asks about in every DS: know a 1-sentence definition + a code example for each

Word	One-sentence definition (course wording)	Tiny example
Class	A family of objects with the same structure (attributes) and behaviour (methods), characterised by a name. It is the "mould".	`public class Car { ... }`
Object	An instance of a class. It has an **identity** (unique), a **type**, a **state** (attribute values at a given time) and a **behaviour** (its methods).	`Car bmw = new Car("X1123","white",110);`
Attribute (instance variable)	A variable declared in the class, outside every method. Each object has its own copy; it is visible in all methods of the class.	`private double speed;`
Local variable	A variable declared inside a method (or block). Only known in that block, lost when the method ends.	`int i = 0;` inside a method
Method	A function defined in a class that manipulates the object's data: `<return type> name(<parameters>) { body }`. `void` = returns nothing.	`public double area() { return 3.14*r*r; }`
Constructor	A special method with the **same name as the class** and **no return type** (not even `void`), called **once**, automatically, at creation (`new`), to initialise the attributes.	`public Car(String m) { model = m; }`
this	A reference to the **current object** (the one running the method). Used to remove ambiguity (`this.x = x`) and to call another constructor (`this(0,0)`).	`this.name = name;`
Encapsulation	Grouping data + methods in the class and hiding the details: attributes `private`, access only through public methods (getters/setters). It guarantees data integrity.	`private double balance;` + `getBalance()`

Overloading	Several methods with the same name in one class, different number and/or types of parameters. Also called **static polymorphism**.	`deposit(int)` / `deposit(double)`

1. Introduction: what Java is (slides 5–11)

OOP in one picture

- We represent real-world things as **objects**.
- A **class** = name + attributes (data) + methods (processing).
- A program = objects interacting. **Data-driven**, not process-driven.

```
+--------------------+
|        Car         |  ← class name
+--------------------+
| registrationNumber |
| color              |  ← attributes
| speed              |
+--------------------+
| increaseSpeed()    |  ← methods
| decreaseSpeed()    |
+--------------------+
```

Compiled AND interpreted

```
MyClass.java  (source code)
     |  javac   (compiler)
     ▼
MyClass.class (bytecode, portable)
     |  java    (JVM on Windows / Linux / Mac)
     ▼
  execution
```

- **Bytecode** = portable pseudo-code produced by `javac`.
- **JVM** (Java Virtual Machine) executes the bytecode, so the same `.class` runs everywhere → **portability**.

Why Java? (slide 8)

Simple · Secure · Robust · Distributed (network API) · Multi-thread · Rich library.

JDK = Java Development Kit (slide 9)

JRE (runtime) · standard libraries (`java.lang`, `java.util` …) · compiler `javac` · interpreter `java` · `appletviewer` · `javadoc` (comments `/** */`) · debugger `jdb`.
IDEs: Eclipse, IntelliJ IDEA, NetBeans, BlueJ.

Structure of a program (slide 11)

```java
// File MUST be called Hello.java (same name as the public class)
public class Hello {
    // attributes declaration
    // methods definition
    public static void main(String args[]) {
        // local variables declaration
        System.out.println("Hello I4.0");
    }
}
```

⚠ **Traps**
Java is **case-sensitive**: `System` ≠ `system`, `public` ≠ `Public`. The file name must match the `public` class name. `main` is the entry point: `public static void main(String[] args)` (`String args[]` is the same thing).

2. Chapter 1: basics (types, variables, operators) (slides 13–22)

Naming conventions & reserved words (slides 14–15)

What	Rule	Example
variables, attributes, methods	start lowercase, then each word capitalised (camelCase)	`i`, `setLength`, `anIntegerVariable`

classes, interfaces	start with a capital letter	`MyClass`, `TheGoodJob`
constants	ALL CAPS, declared `final`, **must be initialised at declaration**	`final double PI = 3.141592;`
reserved words	can't be used as identifiers	`abstract, boolean, do, if, package, null, goto, double…`

Primitive types

Type	Holds	Example literal
`byte` / `short`	small integers	`(byte) 10`
`int`	integer (default)	`27`
`long`	big integer	`27L`
`float`	decimal (less precise)	`1.5f` ← the **f** is required
`double`	decimal (default)	`1.5`
`char`	one character	`'x'` (single quotes)
`boolean`	true / false	`true`

`String` ("hello", double quotes) is **not** primitive: it is a class (capital S).

Declaring variables (slide 19)

```
// TypeName variableName = value;   or
TypeName variableName;
String greeting = "Hello, Dave!";
int age = 20;
double price = 9.99;
char grade = 'A';
boolean isOpen = true;
final int MAX_SPEED = 120;   // constant
int a, b, c;                 // several at
once
a = b = c = 0;
```

Type conversion (slides 17–18): exam favourite

Implicit (automatic, safe = widening):

```
byte → short → int → long → float → double
              char → int → long → float →
double
```

Explicit (cast) required when you can lose information (e.g. `float`/`double` → `int`):

```
int i, j;
short s = 2;
float f = 1.5f;       // slide 18 writes 1.5 →
compile error!
i = (int) f;          // explicit cast: i = 1
(decimals cut)
i = s;                // implicit: short → int
double d = i / 2.0;   // arithmetic promotion:
0.5
```

⚠ THE integer-division trap (slide 21, Lab 1 Ex1)

int ÷ int = int: the decimal part is **cut** (not rounded), even if you store the result in a double.

```
int a = 4, b = 7;
double m;
m = (a + b) / 2;          // 11/2 = 5   → m =
5.0  ✗
m = (a + b) / 2.0;        // 5.5  ✓ (one
operand is double)
m = (double) (a + b) / 2; // 5.5  ✓ (cast)
```

```
double celsius = 25;
double f1 = celsius * 9 / 5 + 32;   // OK
here: celsius is double
double f2 = 9 / 5 * celsius + 32;   // ✗ 9/5
= 1 → 57.0
double f3 = 9.0 / 5 * celsius + 32; // ✓ 77.0
```

Rule: evaluation goes left to right; as soon as an int/int division happens, the decimals are lost.

Operators

Kind	Operators	Notes

Arithmetic	`+ - * / %`	`%` = remainder: `7 % 2 = 1`, `year % 4 == 0` → divisible by 4
Increment	`++ --`	`i++` (use then add) vs `++i` (add then use)
Compound	`+= -= *= /=`	`result *= i;` ⇔ `result = result * i;`
Comparison	`== != < > <= >=`	result is `boolean`. `=` assigns, `==` compares!
Logical	`&&` (and) `||` (or) `!` (not)	`!(a < c)`
String concat	`+`	`"Sum: " + 2 + 3` → `"Sum: 23"`; `"Sum: " + (2 + 3)` → `"Sum: 5"`

Slide 20, Exercise 1: what does it print?

```
int a, b, c;
boolean resultat;
a = 3; b = 3; c = 4;
resultat = (a > b);
System.out.println(resultat);    // false
resultat = (a != c);
System.out.println(resultat);    // true
System.out.println(a != c);      // true
System.out.println(a >= b);      // true
System.out.println(!(a < c));    // false
```

Slide 22, Exercise 3: swap two integers

```
int a = 5, b = 8;
int temp = a;   // 1. save a
a = b;          // 2. a gets b
b = temp;       // 3. b gets old a
System.out.println("a = " + a + ", b = " + b);
// a = 8, b = 5
```

Slide 32 asks the same with `a` and `b` read from the keyboard: see the Scanner section.

Printing

```
System.out.println("with newline");
System.out.print("no newline ");
System.out.println("x = " + 5);
System.out.printf("%.2f%n", 3.14159);   // (extra) prints 3.14
```

3. Control structures & Scanner (slides 23–33, Lab 1)

if / else if / else

```
int variable = 15;
if (variable == 20) {
    System.out.println("equal to 20");
} else if (variable == 15) {
    System.out.println("not 20, equal to 15");
} else {
    System.out.println("neither 20 nor 15");
}
```

Lab 1 Ex2, leap year: divisible by 4 but not by 100, OR divisible by 400.

```
int year = 2024;
if ((year % 4 == 0 && year % 100 != 0) || year
% 400 == 0) {
    System.out.println("Leap Year");
} else {
    System.out.println("Not a Leap Year");
}
```

switch (slide 28, Lab 1 Ex3)

```
char op = '+';
double x = 5, y = 3;
switch (op) {
    case '+': System.out.println(x + " + " + y
+ " = " + (x + y)); break;
    case '-': System.out.println(x - y);
break;
    case '*': System.out.println(x * y);
break;
    case '/':
        if (y != 0) System.out.println(x / y);
        else System.out.println("Division by
zero!");
        break;
    default:  System.out.println("Invalid
operator");
}
```

> ⚠ **switch rules**
>
> - Course: works on simple types `int, short, char, byte` (not `double`, not `boolean`). (extra: Java 7+ also allows `String`.)
> - `break` = exit the switch. **Forget it → "fall-through":** the next cases run too.
> - `default` = runs if no case matches.

Loops

	`for`	`while`	`do … while`
Use when	number of repetitions **known** (determinate)	repeat **while** a condition is true (indefinite)	indefinite, but must run **at least once**
Condition tested	before each turn	before each turn → may run **0 times**	**at the end** → runs ≥ 1 time
Lab example	multiplication table (Ex4)	input validation	guessing game (Ex5), menu

```java
// for: Lab 1 Ex4, multiplication table of n
int n = 5;
for (int i = 1; i <= 10; i++) {
    System.out.println(n + " x " + i + " = " +
(n * i));
}
// while
int k = 1;
while (k <= 3) {
    System.out.println(k);
    k++;              // don't forget, or
infinite loop!
}
```

```java
// do...while: Lab 1 Ex5, guessing game
final int SECRET_NUMBER = 42;
int guess, attempts = 0;
do {
    System.out.print("Your guess: ");
    guess = sc.nextInt();
    attempts++;
    if (guess > SECRET_NUMBER)
System.out.println("Too high");
    else if (guess < SECRET_NUMBER)
System.out.println("Too low");
} while (guess != SECRET_NUMBER);
System.out.println("Bravo! Found in " +
attempts + " attempts");
```

The Scanner class (slides 30–31): reading from the keyboard

```java
import java.util.Scanner;     // or import java.util.*;  (1st line!)

public class ScannerExample {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = in.nextLine();
        System.out.print("Enter a and b: ");
        int a = in.nextInt();
        int b = in.nextInt();
        System.out.print("Price: ");
        double p = in.nextDouble();
        System.out.print("Operator: ");
        char op = in.next().charAt(0);   // read a char
        System.out.println("Name is: " + name);
        in.close();
    }
}
```

Method	Reads
`nextInt()`	an `int`
`nextDouble()`	a `double`
`next()`	one word (`String`, stops at space)
`nextLine()`	the whole line (`String`)
`next().charAt(0)`	a `char` (there's no `nextChar()`)
`nextBoolean()`	a `boolean`

⚠ Scanner traps

- Forgetting `import java.util.Scanner;` → "cannot find symbol Scanner".
- Package: `java.util` (QCM!). Available since JDK 1.5.
- (extra) `nextLine()` right after `nextInt()` returns an empty string (it reads the leftover Enter). Add one extra `in.nextLine();` to clear it.
- Close it at the end: `in.close();`

Slide 33, Exercise 4: factorial (the prof's solution, shortened)

```java
import java.util.Scanner;

public class Factorial {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a natural number N: ");
        int n = input.nextInt();
        if (n < 0) {
            System.out.println("Error: Please enter a non-negative integer.");
        } else {
            long result = 1;                 // long: factorials grow fast
            for (int i = 1; i <= n; i++) {
                result *= i;
            }
            System.out.println(n + "! = " + result);
        }
        input.close();
    }
}
```

4. Chapter 2: class, object, attributes, methods (slides 35–51)

Object (slides 36, 49)

An entity manipulated in a program by calling its methods. It has:

- **Identity**: unique, distinguishes it from all others (it can be named/referenced).
- **Type**: its class.
- **State**: values of its attributes at a given time.
- **Behaviour**: its methods (consultation = getters, update = setters).

An object is an instance of a **single** class; any method of that class can be applied to it.

Class (slide 37)

A family of objects with the same structure and behaviour, with a name. Lets you create (instantiate) as many objects as you want. Convention: `ClassName objectA`.

Class example (slides 38–40)

```java
public class Person {
    // attributes: visible in ALL methods of the class
    public String name, lastName;
    public int age;

    // method that returns nothing → void
    public void setName(String aName) {
        name = aName;
    }
    // method that returns a String
    public String getName() {
        return name;
    }
}
```

Declaring a method (slides 40–47)

Syntax

`[public/private/protected] [static] <return type> methodName(<type1 p1, type2 p2, …>) { <method body> }`

- **Return type**: a simple type or a class name: `double min(double a, double b)`, `int[] first(int n)`, `Color getColor()`.
- `void` if it returns nothing: `void display(double[][] m)`.
- **Parameters**: empty `()` or comma-separated *type name* pairs.
- If the return type isn't `void`, the body **must** contain a `return expression;` compatible with the type. Several `return`s are allowed.
- In a `void` method, `return;` just exits early.

```java
// several returns
double min(double a, double b) {
    if (a < b) return a;
    return b;
}
// return; to leave a void method early (slide 45)
void showPosition(int[] tab, int val) {
    for (int i = 0; i < tab.length; i++) {
        if (tab[i] == val) {
            System.out.println("Position of " + val + " is " + i);
            return;
        }
    }
    System.out.println(val + " is not in the array");
}
```

Local variables (slides 46–47)

Declared inside a method; store the data the method uses; only accessible in their block; their value is **lost** when the method ends. Two different methods can use the same local name (`int i` in both) without conflict. Two different classes can have members with the same name (`Point.x` and `Circle.x`): Java knows which one from the object's type.

Creating objects = instantiation (slides 50–51)

3 steps:

1. declare a **reference** variable of the class type,
2. `new` allocates memory,
3. a **constructor** creates and initialises the object.

Rule

```
ObjectType objectName = new Constructor(parameters);
```

```
MyClass myObject = new MyClass();

Person john, peter, x;   // 3 references, NO objects yet
john = new Person();     // now an object exists
peter = new Person();
x = new Person();

john.setName("John");           // dot "." to access members
System.out.println(john.getName());
```

⚠ **Reference ≠ object**

`Circle c;` creates only a reference (no object). `Person p2 = p1;` does NOT copy the object: both names point to the **same** object, so `p2.age = 30` also changes `p1.age`.

5. Constructors & object destruction (slides 52–62)

Constructor: the 6 rules (slide 52), learn them as a list

1. Special method called **only once per object**, at creation (`new`).
2. Has **the same name as the class**.
3. Role: **initialise the attributes**.
4. Generally `public`, **no return type, not even** `void`.
5. A class can have **several** constructors, differing in number/type of parameters (= overloading).
6. If you write **no** constructor, Java gives a **default constructor** with no parameters that sets attributes to `0` (numbers), `null` (references/String), `false` (boolean). **As soon as you write one, the default one disappears.**

```java
public class Person {
    public String name;
    public int age;

    public Person(int x) {              // constructor 1
        age = x;
    }
    public Person(String s) {           // constructor 2
        name = s;
    }
    public Person(String s, int x) {    // constructor 3
        name = s;
        age = x;
    }
}
```

```java
Person p1 = new Person(10);         // → constructor 1
Person p2 = new Person("Ali", 10); // → constructor 3
Person p3 = new Person("Ali");      // → constructor 2
Person p4 = new Person();           // ✗ COMPILE ERROR:
// no constructor without parameters exists any more
```

⚠ Slide 54 trap

Before you add `Person(String, String, int)`, `new Person()` works (default constructor). After you add it, the same line is a compilation error. Fix: also write `public Person() { }`.

Slide 56: the Circle example

```java
class Circle {
    public double x, y;     // centre
    private double r;       // radius

    public Circle(double r1) {
        r = r1;
    }
    public double area() {
        return 3.14159 * r * r;
    }
}

public class MyProgram {
    public static void main(String[] args) {
        Circle c;              // c is a reference, not an object
        c = new Circle(5);     // c now references an object in memory
        c.x = c.y = 10;
        System.out.println("Area of c: " + c.area());
    }
}
```

Exercise slide 62: Time (3 constructors, very likely in the DS)

```java
public class Time {
    private int h, m, s;

    public Time(int hour) {
        this(hour, 0, 0);                  // reuse the 3-parameter constructor
    }
    public Time(int hour, int minute) {
        this(hour, minute, 0);
    }
    public Time(int hour, int minute, int second) {
        h = hour;
        m = minute;
        s = second;
    }
    public void display() {
        System.out.println("It is " + h + " hours " + m + " minutes " + s + " seconds");
    }
    public static void main(String[] args) {
        Time t = new Time(14, 30);
        t.display();                       // It is 14 hours 30 minutes 0 seconds
    }
}
```

Exercise slide 59: Book

```java
public class Book {
    private String title, author;
    private double price;

    public Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
    }
    public void displayDetails() {
        System.out.println("Title: " + title + ", Author: "
                           + author + ", Price: " + price);
    }
    public static void main(String[] args) {
        Book b1 = new Book("Dune", "Herbert", 25.5);
        Book b2 = new Book("1984", "Orwell", 18.0);
        b1.displayDetails();
        b2.displayDetails();
    }
}
```

Exercise slide 58: Rectangle

```java
public class Rectangle {
    private float width, length;

    public Rectangle(float width, float length) {
        this.width = width;
        this.length = length;
    }
    public float perimeter() { return 2 * (width + length); }
    public double diagonal() {
        return Math.sqrt(width * width + length * length);
    }
    // the slide calls it double(): illegal, double is a reserved word
    public void doubleLength() { length = length * 2; }
}
```

Circle (slide 57): `2 * Math.PI * radius` and `Math.PI * radius * radius`.

Object destruction: Garbage Collector (slides 60–61)

- Other languages have a **destructor** that frees the resources the constructor allocated.
- In Java it's **automatic**: the **Garbage Collector (GC)**.
- An object can be destroyed when **no reference points to it any more** (e.g. `obj = null;`).
- You can ask for it explicitly with `System.gc();` (the JVM decides when it really runs).

```
Date dt = new Date(1, 6, 1997);
dt.show();
dt = null;      // the object has no reference any more:
                // it's unreachable, the GC may destroy it
System.gc();    // ask the GC to run
```

6. The keyword `this` (slides 63–64, Lab 3 Ex4–5)

`this` = reference to the **current object**, the object whose method is running.

Use 1: remove ambiguity (parameter hides attribute)

```
public Point(double x, double y) {
    x = x;      // ✗ assigns the parameter to ITSELF
    y = y;      //   the attributes stay 0.0
}
```

```
class Point {
    double x, y;
    public Point(double x, double y) {
        this.x = x;   // ✓ attribute = parameter
        this.y = y;
    }
}
```

Use 2: call another constructor: `this(...)`

```
class Point {
    double x, y;
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
    public Point() {
        this(0, 0);   // MUST be the FIRST statement
    }
}
```

Use 3: return the current object (method chaining, Lab 3 Ex4)

```java
class Student {
    int id;
    String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
    public Student incrementAndReturnId() {
        id++;
        return this;        // returns the object itself
    }
    public void display() {
        System.out.println("Student ID: " + id + ", Name: " + name);
    }
}
public class TestStudent {
    public static void main(String[] args) {
        Student s1 = new Student(7, "Amine");
        s1.incrementAndReturnId().display(); // Student ID: 8, Name: Amine
    }
}
```

Lab 3 Ex5: default constructor chained with `this(...)`

```java
class Car {
    String model, fuelType;
    int year;

    public Car() {
        this("Unknown Model", "Petrol", 2023); // first line!
        System.out.println("Default constructor called.");
    }
    public Car(String model, String fuelType, int year) {
        this.model = model;
        this.fuelType = fuelType;
        this.year = year;
    }
    public void displayCar() {
        System.out.println("Car Model: " + model + " | Fuel: " + fuelType + " | Year: " + year);
    }
}
```

⚠ **`this` traps**

`this(...)` anywhere other than the first line of a constructor → compile error. `this` can't be used inside a `static` method (e.g. `main`), because there's no current object there.

7. Access modifiers & encapsulation (slides 65–77, Lab 3 Ex1–2)

Full declaration syntax (slides 65–69)

Class
`[public] [abstract/final] class ClassName [extends SuperClass] [implements I1, I2…] { attributes; methods; }`
Attribute `[private/public/protected] [static] [final] type name [= value];`
Method `[private/public/protected] [static] [final/abstract] returnType name([type p1, …]) { … [return result;] }`

Keyword	On an attribute	On a method
`static`	only **one copy**, shared by all objects (class attribute)	not tied to an object: called with `ClassName.method()` (class method)
`final`	constant: can't be changed	can't be redefined in a subclass (later chapter)
`abstract`	-	no body; the class becomes abstract (later chapter)
`extends` / `implements`	inheritance / interfaces: next chapters	

Visibility table (slide 71): learn it by heart

Accessible to →	same class	subclass, same package	class, same package	subclass, other package	class, other package
`public`	✓	✓	✓	✓	✓
`protected`	✓	✓	✓	✓	✗
friendly (no keyword, package)	✓	✓	✓	✗	✗
`private`	✓	✗	✗	✗	✗

Slide 70 example: `c1` has `public int A; protected int B; int C; private int D;`. `c2 extends c1` and `c3` are in package P1 (same as c1); `c4 extends c1` and `c5` are in P2.

	A	B	C	D
c2 (sub, P1)	✓	✓	✓	✗
c3 (P1)	✓	✓	✓	✗
c4 (sub, P2)	✓	✓	✗	✗
c5 (P2)	✓	✗	✗	✗

Memory trick: from most open to most closed: `public` > `protected` > friendly > `private`. private = "only me" · friendly = "my package" · protected = "my package + my children" · public = "everyone".

Class modifiers (slide 66)
`public class` = usable by all classes. With **no modifier** = usable only inside its package. (The slide says "`private`" for this; see errata #6: the compiler refuses `private` on a top-level class.)

Encapsulation (slide 76)

Definition
Encapsulation brings data and methods together in one structure (the class). Implementation details are **hidden**: the outside world accesses the data only through a set of operations (the object's **interface**: public methods). It **guarantees the integrity** of the object's data.

Lab 3 Ex1: why it doesn't compile

```
class BankAccount {
    private String accountNumber;
    private double balance;
}
// in main, another class:
myAccount.accountNumber = "ACC-100203"; // ERROR 1
myAccount.balance = 500.0;              // ERROR 2
```

Why: the attributes are `private`, so they're only accessible inside `BankAccount`'s own methods. `main` is in another class → "has private access" compile error.

The fix: getters/setters (+ validation)

```
class BankAccount {
    private String accountNumber;
    private double balance;

    public String getAccountNumber() { return accountNumber; }
    public void setAccountNumber(String accountNumber) {
        this.accountNumber = accountNumber;
    }
    public double getBalance() { return balance; }   // no setBalance!
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        } else {
            System.out.println("Invalid amount");
        }
    }
}
public class Ex1_Encapsulation {
    public static void main(String[] args) {
        BankAccount myAccount = new BankAccount();
        myAccount.setAccountNumber("ACC-100203");
        myAccount.deposit(500.0);
        System.out.println("Account Number: " + myAccount.getAccountNumber());
        System.out.println("Current Balance: $" + myAccount.getBalance());
    }
}
```

🎯 Getter / setter pattern (write it without thinking)
`public Type getX() { return x; }` · `public void setX(Type x) { if (valid) this.x = x; }` · for a boolean: `isX()`. Why it's better than a `public` attribute: the setter can **refuse invalid values** (negative balance, battery > 100...), and you can make an attribute read-only (getter without setter).

8. Scope of variables (slides 72–73)

```
public class AClass {
    int i, j, k;                  // attributes: known everywhere in the class

    public void sth1(int z) {
        int j, r;                 // local j HIDES the attribute j
        r = z;
        j = z;                    // → the LOCAL j
    }
    public void sth2() {
        k = r;                    // ✗ ERROR: r only exists in sth1
    }
}
```

- A variable is known **only in the block `{ }`** where it is declared (and nested blocks).
- Attributes (declared in the class) are known in every method.
- Name conflict local vs attribute → **the most local one wins**. Use `this.j` to reach the attribute.
- `for (int i = 0; …)` : `i` dies at the end of the loop.

	Attribute	**Local variable**
Declared	in the class	in a method/block
Lifetime	as long as the object	until the block ends
Default value	0 / false / null	**none**: must be initialised before use

9. Method overloading (slides 74–75)

- **Definition:** several methods with the **same name** in one class.
- They must differ by the **number** and/or **types** of parameters.
- The return type may differ, but **never as the only difference**.
- Several constructors = a special case of overloading.
- Also called **static polymorphism** (the compiler chooses the method from the arguments).

```
int f(int x)    { ... }
double f(int x) { ... }   // ✗ only the return type differs
```

```
int solde;
public void deposit(int amount) {
    solde += amount;
}
public void deposit(double amount) {
    solde += (int) amount;
}
public void deposit(int amount, String msg) {
// different count
    solde += amount;
    System.out.println(msg);
}
```

Lab 2 Ex2: `calculateExpressCost(boolean isExpress)` is described as "an overloaded version" of `calculateBaseCost()` (same idea, different parameters).

10. static and final (slides 67, 69)

```java
public class Counter {
    static int count = 0;      // ONE copy shared by all objects
    final int ID;              // constant per object
    public static final double PI = 3.14159; // class constant

    public Counter() {
        count++;
        ID = count;            // a final attribute is set once
    }
    static int square(int x) { // class method: no object needed
        return x * x;
    }
    public static void main(String[] args) {
        new Counter();
        new Counter();
        Counter c3 = new Counter();
        System.out.println(Counter.count);
// 3
        System.out.println(c3.ID);
// 3
        System.out.println(Counter.square(4));
// 16
    }
}
```

- `static` **attribute** = class attribute: one value for the whole class, e.g. a counter of created objects. Access: `ClassName.attr`.
- `static` **method** = class method: called without an object (`Math.sqrt(x)`, `Math.random()`, `main`).
- A static method **can't** use non-static attributes or `this` directly. That's why `main` must create an object first: `Time t = new Time(5); t.display();`
- `final` = constant; must be initialised (at declaration, or in the constructor for an attribute). Re-assigning it → compile error.

11. Lab patterns you must be able to rewrite (Labs 2 & 3)

Pattern A: "initialize" method instead of constructor (Lab 2 Ex1 Thermostat, Ex3 FitnessTracker)

```
public class Thermostat {
    String roomName;
    double currentTemperature, targetTemperature;

    void initializeSettings(String name, double target) {
        roomName = name;
        targetTemperature = target;
        currentTemperature = 20.0;       // default value
    }
    void applyHeat(double amount)    { currentTemperature += amount; }
    void applyCooling(double amount) { currentTemperature -= amount; }
    void displayStatus() {
        System.out.println(roomName + ": " + currentTemperature
                           + " / target " + targetTemperature);
        if (currentTemperature < targetTemperature)
            System.out.println("[Status: HEATING ACTIVE]");
        else
            System.out.println("[Status: IDLE]");
    }
    public static void main(String[] args) {
        Thermostat t = new Thermostat();
        t.initializeSettings("Living Room", 24.0);
        t.displayStatus();
        t.applyHeat(2.5);
        t.displayStatus();
    }
}
```

```
// FitnessTracker: validation inside a method
String userName;
int stepsCount;
double caloriesBurnedPerStep;

void walk(int steps) {
    if (steps < 0) {
        System.out.println("Invalid movement!");
        return;                        // data unchanged
    }
    stepsCount += steps;
}
double getTotalCalories() {
    return stepsCount * caloriesBurnedPerStep;
}
```

Scanner inside a method (Lab 2 Ex2)

```
void inputPackageData() {
    Scanner sc = new Scanner(System.in);
    System.out.print("Name: ");
    packageName = sc.nextLine();
    System.out.print("Weight (kg): ");
    weight = sc.nextDouble();
    System.out.print("Distance (km): ");
    distance = sc.nextInt();
}
double calculateBaseCost() {
    return weight * 1.5 + distance * 0.20;
}
double calculateExpressCost(boolean isExpress) {
    if (isExpress) return calculateBaseCost() + 15.0;
    return calculateBaseCost();     // a method calling another method
}
```

Pattern B: an object inside another object = composition (Lab 3 Ex3)

```
class Address {
    private String street, city;
    public Address(String street, String city) {
        this.street = street;
        this.city = city;
    }
    public void showAddress() {
        System.out.println("Delivery Address: " + street + ", " + city);
    }
}
class Customer {
    private String customerName, phoneNumber;
    private Address address;          // attribute of type Address
    public Customer(String n, String p, Address a) {
        customerName = n;
        phoneNumber = p;
        address = a;
    }
    public void displayCustomerProfile() {
        System.out.println("Customer: " + customerName);
        System.out.println("Phone: " + phoneNumber);
        address.showAddress();        // call a method of the inner object
    }
}
public class Ex3_ObjectAccess {
    public static void main(String[] args) {
        Address a = new Address("5th Avenue", "Tunis");
        Customer c = new Customer("Med Abdallah", "+216 98 123 456", a);
        c.displayCustomerProfile();
    }
}
```

Pattern C: array of objects + `Math.random()` (Lab 2 slot machine)

```
class Wheel {
    private int symbol;
    public void spin() {
        // Math.random() gives a double in [0, 1)
        symbol = (int) (Math.random() * 3) + 1;   // 1, 2 or 3
    }
    public int getSymbol() { return symbol; }
}
class BanditManchot {
    private Wheel[] wheels;
    public BanditManchot() {
        wheels = new Wheel[3];           // array of 3 REFERENCES (null)
        for (int i = 0; i < wheels.length; i++)
            wheels[i] = new Wheel();     // create each object!
    }
    public int play() {
        for (int i = 0; i < 3; i++) {
            wheels[i].spin();
            System.out.print(wheels[i].getSymbol() + " ");
        }
        System.out.println();
        int a = wheels[0].getSymbol(), b = wheels[1].getSymbol(),
            c = wheels[2].getSymbol();
        if (a == b && b == c) return 100;
        if (a == b || b == c || a == c) return 10;
        return 0;
    }
}
public class TestJeu {
    public static void main(String[] args) {
        int score = new BanditManchot().play();
        if (score == 100) System.out.println("You've won! Your score is 100 points");
        else if (score == 10) System.out.println("You're about to succeed! Your score is 10
points");
        else System.out.println("Sorry! See you next time.");
    }
}
```

> ⚠ **Arrays of objects**
> `new Wheel[3]` creates 3 empty boxes (`null`), not 3 wheels. Calling `wheels[0].spin()` before `wheels[0] = new Wheel()` → `NullPointerException`.
> Random int in [min, max]: `(int)(Math.random() * (max - min + 1)) + min`.

Arrays: quick syntax

```
int[] t = new int[5];              // 5 zeros
int[] u = {4, 8, 15};              // initialised
System.out.println(u.length);      // 3 (no parentheses: it's an attribute)
for (int i = 0; i < u.length; i++) System.out.print(u[i] + " ");
System.out.println();
double[][] m = new double[2][3];  // 2 rows, 3 columns
m[1][2] = 7.5;
```

12. Short answers to learn by heart (questions de cours)

Question	Answer to write

What is OOP?	Representing real-world elements as objects (data + processing) with a high level of abstraction; a program is a set of interacting objects. Data-driven, not process-driven.
Why is Java portable?	`javac` compiles the source into **bytecode**, which the **JVM** executes; a JVM exists for each OS, so the same bytecode runs everywhere.
JDK vs JRE vs JVM	JVM executes bytecode; JRE = JVM + standard libraries (to run); JDK = JRE + development tools (javac, javadoc, jdb…) (to develop).
Class vs object	Class = mould/model (structure + behaviour); object = a concrete instance of the class with its own state and identity.
Attribute vs local variable	Attribute: declared in the class, known in all methods, lives with the object, default value. Local: declared in a method, known only in its block, lost at the end, no default value.
What is a constructor? Differences with a method?	Same name as the class, no return type, called automatically once at creation by `new`, initialises attributes. A method has a return type (or void) and can be called many times.
Default constructor?	Provided by Java only if no constructor is written; no parameters; sets 0/false/null. Disappears as soon as one constructor is defined.
Uses of `this`?	(1) access an attribute hidden by a parameter (`this.x = x`), (2) call another constructor of the same class, first statement (`this(0,0)`), (3) return/pass the current object (`return this;`).
Encapsulation?	Data + methods in the class, details hidden (`private`), access only through public methods → integrity of the data.
Overloading?	Same method name, different parameter list (number/types). Return type alone isn't enough. = static polymorphism.
`static`?	Belongs to the class, not to an object: one shared copy (attribute) / callable without an object (method).
Garbage collector?	Automatic mechanism that frees the memory of objects no longer referenced (`obj = null`); can be requested with `System.gc()`.
while vs do…while	while tests first (0 or more turns); do…while tests at the end (at least 1 turn).

13. Compile errors she loves to put in "find the error" questions

Code	Error	Fix
`float f = 1.5;`	lossy conversion double → float	`1.5f` or `(float) 1.5`
`int x = 2.5;` / `int i = f;`	lossy conversion	`(int) 2.5` or use `double`
`p.balance = 5;` (private)	"has private access"	getter/setter
`new Person()` when only `Person(String)` exists	constructor cannot be applied	add `public Person() {}` or pass arguments
`public void Person(...)`	not a constructor any more, it's a method!	remove `void`
non-void method without `return`	missing return statement	add `return value;`

`this(...)` not on the first line	call to this must be first statement	move it up
two methods differing only by return type	method already defined	change the parameters
re-assigning a `final`	cannot assign a value to final variable	don't, or remove `final`
using a local variable of another method / before init	cannot find symbol / might not have been initialized	declare/initialise it in the right block
`Scanner` without import	cannot find symbol	`import java.util.Scanner;`
`system.out`, `Public`, `String` → `string`	case-sensitive	correct the case
`x = x;` in a constructor	compiles but attribute stays 0/null (logic bug)	`this.x = x;`
`m = (a+b)/2;` with int a, b	compiles but wrong result (5.0 not 5.5)	`/ 2.0` or cast

14. ⚠ Errata: mistakes in the slides/labs (don't copy them)

1. **Slide 18**: `float f = 1.5;` doesn't compile (1.5 is a double) → `float f = 1.5f;`
2. **Slide 24**: `public int variable ==20;` → `= 20` (== compares, = assigns).
3. **Slide 27**: `system.out.println(i)` → `System.out.println(i);` (capital S + semicolon).
4. **Slide 41/42**: `setColor(color c)` → by convention the class is `Color`.
5. **Slide 55**: `Public Person(...)` → `public` (lowercase, Java is case-sensitive).
6. **Slide 66**: "`private` = class restricted to its package; default is private". Real Java: a top-level class is `public` or has **no modifier** (package/friendly). `private class X` at top level doesn't compile. If asked: "public = all classes; no modifier = same package only".
7. **Slide 56**: `class Circle()` and `public class MyProgram()` → no parentheses after a class name.
8. **Slide 58**: a method named `double()` is illegal (`double` is a reserved word) → `doubleLength()`.
9. **Slide 61**: comment says "Implicit call from the GC"; `System.gc()` is an **explicit** request (the JVM decides if it runs).
10. **Slide 77**: `main(String args {})` → `String args[]`; `new Parallellogramme()` (typo, 2 l's) → `Parallelogramme`.
11. **Lab 2**: `Math.Random()` → `Math.random()` (lowercase r), otherwise compile error.
12. **Slide 15**: "pachage" → `package`.

Next: do the mock DS (`Java_DS_Blanc_Mock_Exam.pdf`) on paper, timed, before looking at the corrigé.